```
var_dump(strpos($quote, 'and'));
var_dump(strrpos($quote, 'and'));
var_dump(strripos($quote, 'AnD'));
var_dump(strpos($quote, 'so long and thanks for all the fish'));
```

```
Output:
int(0)
int(0)
int(6)
int(41)
int(41)
bool(false)
```

The code above demonstrates all of the variants of `strpos()` (`stripos()`, `strrpos()`, and `strripos()`). The sub-string `'We demand'` exists at the very beginning of the `$quote` string; the call to `strpos()` on line three returns an integer position of `0`. If we accessed using array syntax, `$quote[0]` has a value of `W`, `$quote[1]` is `e`, and so on. The next line calls the case-insensitive form `stripos()`. (Remember that this will only work for ASCII characters!) with the same result of `0`. The next two lines call `strpos()` and `strrpos()`, searching for the sub-string `'and'`, which is found twice in the `$quote` string. `strpos()` returns `6` (the first time the sub-string is found), and `strrpos()` – which is searching backwards from the end of `$quote` returns `41` (what a shame it wasn't `42`).

The call to `strripos()` on line seven demonstrates the case-insensitive form of `strrpos()`, and finally we have a call to `strpos()` looking for a sub-string that does not exist in `$quote`. Pay careful attention here: the return value is boolean `false`, not an integer, but the `str*pos()` functions can also return integer `0`, which will evaluate to boolean `false` in a weakly-typed comparison. *Always* use a type-strict test with the results of `str*pos()`, the expression `(strpos($str, $subStr) == false)` – note the weakly-typed equivalence operator – will evaluate to true if `$subStr` isn't found, and also if it is found at the beginning of the string.

All `str*pos()` functions also accept an optional integer third parameter as an offset: the functions will skip the first (or last, in the case of the reversed forms) N bytes before beginning the search.